
**TRABAJO PRACTICO INTEGRADOR
SISTEMA DE SEMAFOROS INTELIGENTES**



**TRABAJO PRACTICO INTEGRADOR
SISTEMA DE SEMAFOROS INTELIGENTES**

UNIVERSIDAD NACIONAL DEL NORDESTE
FACULTAD DE CIENCIAS EXACTAS Y NATURALES Y AGRIMENSURA (FACENA)



Grupo: 34

Integrantes:

Aguirre Lucas

Morales Francisco

Maidana Fornazarich Timoteo David.

TRABAJO PRACTICO INTEGRADOR

SISTEMA DE SEMAFOROS INTELIGENTES

1. Introducción y Contextualización

El presente informe técnico expone el diseño, desarrollo e implementación del núcleo lógico para un Sistema de Semáforos Inteligentes urbano, diseñado para optimizar de manera automatizada el flujo vehicular y garantizar la consistencia en intersecciones críticas.

El análisis metodológico se divide en dos fases fundamentales: en primer lugar, la codificación rigurosa en el lenguaje puramente funcional Common Lisp, aplicando restricciones estrictas de inmutabilidad; en segundo lugar, una contrastación paradigmática con Scala.

Requerimiento 1: Estados de Transición

Modelado del cambio de luces del semáforo. Recibe el color actual y el destino, devolviendo una lista con la acción o una acción predeterminada si no es válida.

```
1  ;; =====
2  ;; FUNCIÓN: Transición
3  ;; NATURALEZA: Pura (no posee efectos secundarios)
4  ;; ESTRATEGIA: Uso de Cond para cambiar los colores
5  ;; IMPACTO: No destructiva
6  ;; =====
7
8  (defun transicion (color-actual cambiar-a)
9    (cond
10      ((and (not (equal color-actual 'en-rojo)) (equal color-actual 'en-amarillo) (equal cambiar-a 'rojo))
11        (list color-actual "cambiar-a-rojo"))
12
13      ((and (not (equal color-actual 'en-amarillo)) (not (equal color-actual 'amarillo)) (equal cambiar-a 'amarillo))
14        (list color-actual "cambiar-a-amarillo"))
15
16      ((and (not (equal color-actual 'en-verde)) (equal color-actual 'en-amarillo) (equal cambiar-a 'verde))
17        (list color-actual "cambiar-a-verde"))
18
19      (t
20        (list color-actual 'accion-por-defecto)
21      )
22    )
23  )
24
```

TRABAJO PRACTICO INTEGRADOR SISTEMA DE SEMAFOROS INTELIGENTES

Requerimiento 2: Temporizador Automático

Automatiza y calcula la luz que debe encenderse en base a un valor matemático de tiempo Unix (timestamp).

```
25 ;; =====
26 ;; FUNCIÓN: Timer
27 ;; NATURALEZA: Impura (Ya que devolvera un color dependiendo del timestamp)
28 ;; ESTRATEGIA: Orden Inferior (implementa cond)
29 ;; IMPACTO: No destructiva
30 ;; =====
31 (defun timer(timestamp)
32   (cond
33     ((< (rem timestamp 216) 90) 'Rojo)
34     ((< (rem timestamp 216) 96) 'Amaraillo)
35     (t 'Verde)
36   )
37 )
```

Requerimiento 3: Sistema de Auditoría

Función encargada de imprimir directamente en la pantalla de la terminal el historial del cambio de luces y el tiempo asociado.

```
39 ;; =====
40 ;; FUNCIÓN: logging
41 ;; NATURALEZA: Impura (Ya que devuelve un mensaje en la pantalla con el tiempo y dos colores)
42 ;; ESTRATEGIA: Orden Inferior (implementa format)
43 ;; IMPACTO: No destructiva
44 ;; =====
45 (defun logging (timestamp color-actual cambiar-a)
46   (format t "Tiempo ~A : la luz ha cambiado de ~A a ~A" timestamp color-actual cambiar-a)
47 )
```

TRABAJO PRACTICO INTEGRADOR SISTEMA DE SEMAFOROS INTELIGENTES

Requerimiento 4: Análisis de Ciclos Semafóricos

Conjunto de funciones para calcular la suma de los tiempos de un ciclo completo y evaluar mediante rangos si la duración se adapta a los estándares.

```
;; =====  
;; FUNCIÓN: duracion-ciclo  
;; NATURALEZA: Pura (Siempre devuelve el mismo valor entero)  
;; ESTRATEGIA: Orden Inferior (utiliza suma)  
;; IMPACTO: No destructiva  
;; =====  
(defun duracion-ciclo ()  
  (+ 90 6 120)  
)  
  
;; =====  
;; FUNCIÓN: recomendacion-ciclo  
;; NATURALEZA: Pura (Ya que solo retorna los simbolos)  
;; ESTRATEGIA: Estructura condicional (utiliza cond para evaluar cada caso )  
;; IMPACTO: No destructiva  
;; =====  
(defun recomendacion-ciclo (duracion-ciclo)  
  (cond  
    ((> 35 duracion-ciclo) 'Bajo)  
    ((> duracion-ciclo 150) 'Alto)  
    (t 'En-Rango)  
  )  
)
```

Requerimiento 5: Planificación Temporal

Realiza operaciones aritméticas de multiplicación y división para obtener la cantidad de ciclos que caben en un determinado lapso de minutos.

```
73      ;; =====  
74      ;; FUNCIÓN: ciclos-por-tiempo  
75      ;; NATURALEZA: Pura (Muestra la cantidad de ciclos)  
76      ;; ESTRATEGIA: Funciones Aritmeticas (Utiliza Suma y Division para el calculo)  
77      ;; IMPACTO: No destructiva  
78      ;; =====  
79      (defun ciclos-por-tiempo (cantidad-minutos duracion-ciclo)  
80        (float (/ (* cantidad-minutos 60) duracion-ciclo))  
81      )
```

TRABAJO PRACTICO INTEGRADOR SISTEMA DE SEMAFOROS INTELIGENTES

Requerimiento 6: Informe de Distribución Temporal

Estructura una lista que expone los porcentajes de tiempo asignados a cada color en relación con la duración total.

```
83      ;; =====
84      ;; FUNCIÓN: distribucion temporal
85      ;; NATURALEZA: Pura (Devuelve los mismos valores)
86      ;; ESTRATEGIA: Orden inferior (Utiliza list)
87      ;; IMPACTO: No destructiva
88      ;; =====
89      (defun distribucion-temporal (duracion-ciclo)
90        (list 'Rojo (float (* 100 (/ 90 duracion-ciclo)))
91              'Amarillo (float (* 100 (/ 6 duracion-ciclo)))
92              'Verde  (float (* 100 (/ 120 duracion-ciclo))))
93      )
```

Iteración 2 - Extensión 1: Intermitencia de Seguridad

Versiones actualizadas de la lógica de control para incorporar e interpretar los nuevos estados de luces intermitentes.

```
95      ;; ===== ITERACION 2 ===== ;;
96
97      ; =====
98      ;; FUNCIÓN: transicion (Actualizado Iteracion 2)
99      ;; NATURALEZA: Pura (Ya que devuelve una lista segun la accion a realizar sin efectos secundarios)
100     ;; ESTRATEGIA: Estructura condicional (implementa cond para evaluar cada caso)
101     ;; IMPACTO: No destructiva
102     ;; =====
103     (defun transicion (color-actual cambiar-a)
104       (cond
105         ((and (not (equal color-actual 'en-rojo)) (equal color-actual 'amarillo-intermitente) (equal cambiar-a 'rojo)) (list color-actual "cambiar-a-rojo"))
106         ((and (not (equal color-actual 'en-amarillo)) (equal cambiar-a 'amarillo-intermitente)) (list color-actual "cambiar-a-amarillo-intermitente"))
107         ((and (not (equal color-actual 'en-verde)) (equal color-actual 'amarillo-intermitente) (equal cambiar-a 'verde)) (list color-actual "cambiar-a-verde"))
108         (t (list color-actual 'accion-por-defecto))
109       )
110     )
```

TRABAJO PRACTICO INTEGRADOR SISTEMA DE SEMAFOROS INTELIGENTES

```
112 ;; =====
113 ;; FUNCIÓN: timer (Actualizada a la Iteración 2)
114 ;; NATURALEZA: Pura (Ya que devolvera siempre el color ante el mismo timestamp)
115 ;; ESTRATEGIA: Estructura condicional y predicado (implementa cond para evaluar cada caso junto con comparaciones y rem)
116 ;; IMPACTO: No destructiva
117 ;; =====
118 (defun timer(timestamp)
119   (cond
120     ((< (rem timestamp 222) 90) 'Rojo)
121     ((< (rem timestamp 222) 96) 'amarillo)
122     ((< (rem timestamp 222) 93) 'amarillo-intermitente)
123     ((< (rem timestamp 222) 213) 'verde)
124     ((< (rem timestamp 222) 219) 'amarillo)
125     (t 'amarillo-intermitente)
126   )
127 )
```

Iteración 2 - Extensión 2: Persistencia de Datos

Funciones destinadas a la apertura de archivos de texto y al recorrido recursivo por la lista de datos para volcar la información externamente.

```
129 ;; =====
130 ;; FUNCIÓN: informe
131 ;; NATURALEZA: Impura (Crea un archivo de texto)
132 ;; ESTRATEGIA: Ejecucion secuencial
133 ;; IMPACTO: No destructiva
134 ;; =====
135
136 (defun informe (datos)
137   (with-open-file (stream "informe-ejecucion-semaforo.txt" :direction :output)
138     (format stream "Informe de Ejecución del Sistema Semafórico~%")
139     (format stream "=====~%")
140     (logging datos stream)
141
142     (format stream "~% --- Fin del Informe ---")))
```

```
;; =====
;; FUNCIÓN: logging
;; NATURALEZA: Impura (Interactua con el exterior)
;; ESTRATEGIA: recursividad de cola (Utiliza una estructura cond)
;; IMPACTO: No destructiva
;; =====
(defun logging (datos stream)
  (cond
    ((null datos) nil)
    ((consp datos) (format stream "~A - Transición: ~A → ~A" (car datos) (second (car datos)) (third (car datos))) (logging (cdr datos) stream))
    (t (logging (cdr datos))))
)
```

Justificación de librerías Fase 2

Se decidió implementar cl-json ya que en la primera etapa del desarrollo, los tiempos de los colores del semáforo eran constantes fijas dentro de las funciones matemáticas. Al externalizar esta configuración a un archivo config.json, logramos un sistema dinámico y escalable: en caso de que en un futuro se desea modificar dichos valores, se puede directamente realizarlo desde el archivo de texto.

Durante el desarrollo y la integración del entorno, nos enfrentamos a algunos errores clásicos de Lisp que logramos diagnosticar y resolver:

- El Error: Al intentar descargar la herramienta tipeando (ql:quickload "cl.json"), el sistema abortaba.
- La Solución: Corregir la sintaxis reemplazamos el punto por el guion medio correspondiente al nombre oficial del repositorio (cl-json).
- El Error: Al intentar cargar el archivo principal con (load "C:\\clisp-2.49\\core.lisp"), el motor arrojaba que el archivo C:\\clisp-2.49\\core.lisp no existía.
- La Solución: Lisp interpreta la barra invertida (\\) de Windows como un "carácter de escape" para textos especiales y la omite. Lo solucionamos barras inclinadas hacia adelante (/), permitiendo que Lisp lea correctamente.

Comparación códigos Fase 3

No usamos clases ni case classes, sino que lo resolvimos directamente con funciones dentro de un object/archivo con @main. Básicamente lo tratamos de forma más funcional.

La idea fue modelar el semáforo como funciones puras (transición y timer) que reciben valores de entrada y devuelven resultados sin guardar estado en ningún lado.

Manipulación de lista

En Scala, métodos como .map o .filter son bastante fáciles de leer porque están muy cerca del lenguaje natural: uno puede ver "mapear esta lista" o "filtrar esto" directamente sobre la colección.

En Common Lisp también se usan funciones de orden superior, pero la sintaxis es un poco menos directa entonces al principio cuesta más leerlo.

Scala resulta más fácil de leer en general, sobre todo para alguien que no está tan acostumbrado a Lisp, porque es más moderno y más cercano a cómo uno piensa las operaciones sobre listas.